

Graph-Preconditioned Estimation of High-Dimensional Fixed-Effect Models

Alexander Fischer and Kristof Schröder

Draft: May 2026

Abstract. The Method of Alternating Projections (MAP) is the standard algorithm for high-dimensional fixed-effect regressions. It is fast when absorbed factors are well connected, but converges slowly on sparse or nearly nested fixed-effect graphs, such as matched employer-employee panels where worker-firm mobility links separate worker and firm effects. The convergence behavior of MAP depends on the mobility pattern linking workers to firms, yet MAP only indirectly makes use of this information by iterating over one fixed effect at a time. That mobility pattern is, however, directly encoded in the matrix of worker-firm match counts, which after a sign change is a Laplacian and admits sparse approximate Cholesky factorization. We propose a graph-preconditioned Krylov solver with a reusable additive Schwarz preconditioner that exploits this structure through local factor-pair subproblems. Benchmarks show MAP remains fastest on dense designs, while graph preconditioning reduces runtime on sparse worker-firm graphs and under strong sorting.

Keywords: high-dimensional fixed effects; alternating projections; preconditioning; matched employer-employee data; computational econometrics

JEL codes: C55; C63; C81; C87; J31

1. Introduction

Fixed-effects regressions are ubiquitous in applied econometrics. Labor economists use worker and firm fixed effects to separate worker heterogeneity from firm wage premia (Abowd et al. 1999; Card et al. 2013); health economists use patient and doctor fixed effects (Molitor 2018); trade economists absorb exporter, importer, product, and time fixed effects (Head and Mayer 2014); and education researchers study student-teacher panels (Chetty et al. 2014).

The standard computational starting point for such regressions is the Frisch-Waugh-Lovell (FWL) theorem (Frisch and Waugh 1933; Lovell 1963). FWL reduces estimation to residualizing the outcome and every regressor of interest against the fixed-effect design, then running a low-dimensional regression on the residualized variables. FWL is popular because it avoids directly forming and inverting the joint Gramian of the regressors, which easily leads to out-of-memory errors for very high-dimensional fixed effects.

The workhorse method for these fixed-effect residualizations is the Method of Alternating Projections (MAP), also known as iterative demeaning or the “Zig-Zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013). Most leading software implementations of fixed effect regression - `reghdfe` in Stata (Correia

2014; 2017), `fixest` (Bergé et al. 2026) in R, or `PyFixest` in Python (PyFixest Developers 2026) use MAP or MAP-based variants with acceleration.

Taking the worker-firm panel as our running example, MAP cycles through the fixed-effect dimensions: subtract worker means, subtract firm means, subtract year means, and repeat until convergence. Because the worker-firm relationship is handled only through successive residual updates - never by solving a local system directly - convergence depends on how well information propagates across the graph recording which workers appear at which firms.

That co-occurrence graph is formalized by the fixed-effect Gramian (Correia 2017). Its off-diagonal blocks record co-occurrences among absorbed factors: which workers appear at which firms, which patients see which doctors, or which families move across which places. In a worker-firm panel, movers create paths between firms, while stayers add observations without connecting firms to one another. When this graph is sparse or poorly connected, some fixed-effect directions are nearly collinear, and MAP can require many iterations to move information across the graph and to finally converge. The same graph features that determine whether worker and firm effects are identified - worker mobility, sparse mover bridges, component structure - also determine the convergence behavior of MAP.

We propose a new preconditioner that directly encodes the co-occurrence graph. More concretely, we build a Schwarz preconditioner from local factor-pair subproblems that use the graph structure of the Gramian: for example, a worker-firm subproblem includes the observed links between workers and firms. A preconditioner is only useful if it approximates the inverse of the co-occurrence Gramian cheaply. We obtain such an approximation by exploiting that, after a sign change, each factor-pair block is a graph Laplacian, which admits cheap approximate inverses following ideas from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025). The preconditioned system is then solved iteratively via a a Krylov solver.¹ In a range of benchmarks against mature implementations of the method of alternating projections, we find that on sparse, poorly connected graphs - the regime where MAP convergence deteriorates - the graph-preconditioned solver delivers substantial speedups, while on dense, well-connected graphs the preconditioner setup cost does not amortize and MAP should remain the natural default.

The rest of the paper is organized as follows. Section 2 sets up the problem of absorbing fixed effects, and Section 3 introduces the AKM model as a running example. Section 4 develops the graph structure of the fixed-effect Gramian, and Section 5 connects this structure to the convergence behavior of MAP. Sections 6 and 7 build up the factor-pair Schwarz preconditioner, starting from a general discussion of preconditioning and culminating in the construction of the novel graph-based preconditioner. Section 8 reports benchmarks on runtime, memory, and numerical equivalence; Section 9 describes the software through which the new algorithm is available; and Section 10 concludes with practical guidance.

¹The seminal Julia implementation of fixed effects regression, `FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026), also uses a Krylov solver - the LSMR (Fong and Saunders 2011) - but only with diagonal preconditioning, which ignores the off-diagonal co-occurrence structure entirely.

²Researchers use different names for this operation: “absorbing fixed effects”, “demeaning”, “residualizing”, or applying the “within transformation”. We use these terms interchangeably throughout.

2. Absorbing Fixed Effects²

We focus on the linear model

$$y = X\beta + D\alpha + \varepsilon, \quad (1)$$

where X contains the regressors of interest, D is the fixed-effect design matrix, and α collects the fixed-effect coefficients. In high-dimensional applications, D can have hundreds of thousands or millions of columns, so forming the full Gramian of $Z = [X \ D]$, storing it, or applying a direct inverse is infeasible. The Frisch-Waugh-Lovell (FWL) theorem provides an elegant solution to this problem.

In a nutshell, FWL states that the coefficient on X can be obtained without explicitly estimating all elements of α in the final regression. Let

$$P_D = D(D'WD)^{-1}D'W \quad (2)$$

denote the weighted projection onto the column space of D , and let $M_D = I - P_D$ be the corresponding residualization operator. Then the coefficient of interest is

$$\hat{\beta} = (X'WM_DX)^{-1}X'WM_Dy. \quad (3)$$

Equivalently, one first residualizes y and every column of X with respect to the fixed effects, and then regresses the residualized outcome on the residualized covariates:

$$\tilde{y} = M_Dy, \quad \tilde{X} = M_DX, \quad \hat{\beta} = (\tilde{X}'W\tilde{X})^{-1}\tilde{X}'W\tilde{y}. \quad (4)$$

For a model with one regressor of interest, this corresponds to three sequential regressions: regress y on the fixed effects D and keep the residual $\tilde{y}$; regress the covariate x on the same fixed effects and keep the residual $\tilde{x}$; then regress $\tilde{y}$ on $\tilde{x}$. FWL states that the coefficient from this third regression is identical to the coefficient on x in the full regression of y on x and D . With several covariates, the second step is repeated for each column of X , so the computational burden is concentrated in repeatedly applying the fixed-effect residual operator M_D .

The remaining task is therefore to apply the same fixed-effect residualization to several right-hand sides. For any such right-hand side μ , either y or one column of X , the fixed-effect fit solves

$$\hat{\alpha}_\mu = \arg \min_{\alpha} \|D\alpha - \mu\|_W^2, \quad (5)$$

where W is a diagonal matrix of weights. The residualized variable is then $\tilde{\mu} = \mu - D\hat{\alpha}_\mu$. The first-order conditions for this auxiliary least-squares problem are

$$D'W(D\hat{\alpha}_\mu - \mu) = 0, \quad (6)$$

or

$$G\hat{\alpha}_\mu = D'W\mu, \quad G = D'WD. \quad (7)$$

Equation 7 is the linear system we actually need to solve: every residualization, of y or of one column of X , reduces to a system in G with a new right-hand side. MAP, LSMR, and the preconditioned solver developed in this paper are all different strategies of solving this linear system. They differ in many aspects, but for our purposes the key axis of variation is how much of the structure of G each one exploits. The sparsity pattern and conditioning of G determine how many iterations MAP needs to converge,

whether a diagonal preconditioner suffices for a Krylov solver, and ultimately which solver is fastest on a given problem. A careful analysis of the structure of G is therefore foundational to the remainder of this paper, and we turn to it in the next section.

3. A Running Example: The AKM Model

For the remainder of the paper we use the AKM model (Abowd et al. 1999) as our running workhorse example. AKM separates persistent worker heterogeneity from firm wage premia using workers who move across firms, and the regression equation of interest is

$$y_{it} = \alpha_i + \psi_{J(i,t)} + \varphi_t + x'_{it}\beta + \varepsilon_{it}, \quad (8)$$

where α_i is a worker fixed effect, $\psi_{J(i,t)}$ is the fixed effect for the firm employing worker i at time t , and φ_t is a time fixed effect.

A key observation is that the AKM specification carries a graph structure. Workers and firms form a bipartite graph whose edges are employment spells. From this we can also read off a firm-to-firm graph in which two firms are linked whenever some worker has been employed at both. Movers therefore create links between firms, while stayers add observations without linking separate firms. The year effect adds a third, low-dimensional factor that interacts with workers and firms through the same observations. Later sections will show that the connectivity of this graph is what governs how quickly MAP can residualize.

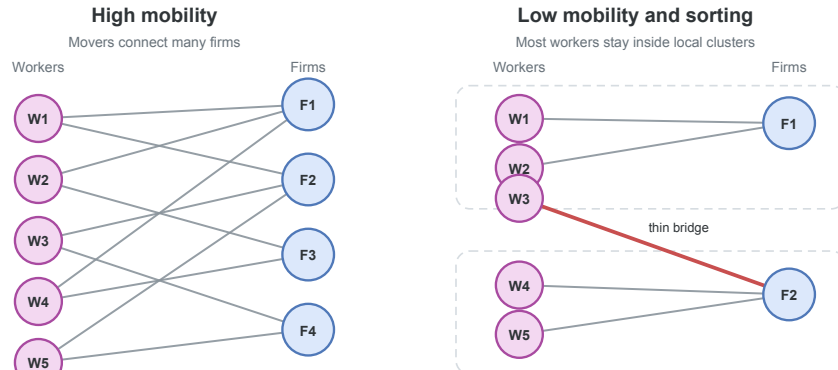


Figure 1: Worker-firm graph connectivity. High mobility creates many paths between firms. Low mobility and sorting leave the graph close to separated clusters connected by thin bridges.

The graph representation links the identification problem to the computational problem. If a worker is only ever observed at one firm, it is difficult to know whether a high wage comes from the worker or the firm, whereas with many workers moving across many firms, each worker carries information across the graph and the two effects are easier to disentangle.

Similar graph interpretations apply to fixed effects models beyond labor economics: doctor-patient panels connect doctors with their patients, student-teacher panels connect students with classrooms, teachers, and schools, and trade datasets connect exporters, importers, products, and years. In each case, the computational difficulty depends on how these fixed-effect levels are connected.

4. The Graph Structure of the Gramian

The bipartite graph introduced in section 3 has an algebraic representation in the block structure of the Gramian $G = D'WD$ (Correia 2017). Suppose the columns of D are ordered as worker levels, firm levels, and year levels. Then

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \quad (9)$$

The **diagonal blocks** G_{WW} , G_{FF} , and G_{YY} contain weighted counts for workers, firms, and years. Since one observation cannot belong to two workers, two firms, or two years at the same time, these blocks are diagonal and can be solved cheaply by dividing by group counts.

The **off-diagonal blocks** are cross-tabulations: the worker-firm block C_{WF} records how often worker i is observed at firm j , and the worker-year and firm-year blocks have analogous interpretations. These blocks encode the cross-factor coupling that MAP touches only through residual updates, while a diagonal LSMR preconditioner uses only the diagonal entries of the Gramian and ignores the cross-factor blocks C_{qr} entirely.

As a small example, we now build a worker-firm panel and populate its Gramian. For simplicity, we set $W = I$.

Obs.	Worker	Firm	Year	y
1	W_1	F_1	Y_1	3.2
2	W_1	F_2	Y_2	4.1
3	W_2	F_1	Y_1	2.8
4	W_2	F_1	Y_2	3.9
5	W_3	F_2	Y_1	5.0
6	W_3	F_2	Y_2	4.5

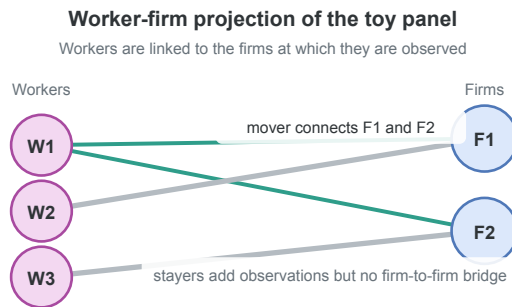


Figure 2: Worker-firm projection of the toy panel. Worker W_1 is a mover; workers W_2 and W_3 are stayers. Worker W_1 works at both F_1 and F_2 and creates a link between the two firms; in AKM terms, W_1 is a mover. Worker W_2 stays at F_1 for two periods, and W_3 stays at F_2 for two periods. Both are stayers. The diagonal blocks are weighted count matrices. In this unweighted example, each worker is observed twice, each firm three times, and each year three times, so

$$G_{WW} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad G_{FF} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}, \quad G_{YY} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}. \quad (10)$$

The off-diagonal blocks are cross-tabulations between factors. The worker-firm block is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \quad (11)$$

The first row says that worker W_1 appears once at firm F_1 and once at firm F_2 . Workers W_2 and W_3 are stayers, appearing twice at F_1 and F_2 , respectively. The worker-year block is

$$C_{WY} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}. \quad (12)$$

Each worker is observed once in each year. The firm-year block is

$$C_{FY} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \quad (13)$$

Firm F_1 appears twice in year Y_1 and once in year Y_2 ; firm F_2 has the opposite pattern. Combining the diagonal count blocks and the off-diagonal cross-tabulation blocks gives the full Gramian.

With column order $(W_1, W_2, W_3, F_1, F_2, Y_1, Y_2)$, the full unweighted Gramian is

$$G = \begin{pmatrix} 2 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 2 & 0 & 2 & 0 & 1 & 1 \\ 0 & 0 & 2 & 0 & 2 & 1 & 1 \\ 1 & 2 & 0 & 3 & 0 & 2 & 1 \\ 1 & 0 & 2 & 0 & 3 & 1 & 2 \\ 1 & 1 & 1 & 2 & 1 & 3 & 0 \\ 1 & 1 & 1 & 1 & 2 & 0 & 3 \end{pmatrix}. \quad (14)$$

The worker-firm block has a direct graph interpretation: worker W_1 is a mover who connects F_1 to F_2 , while workers W_2 and W_3 are stayers. Thus, in the worker-firm part of the graph, the information separating the two firm effects travels through W_1 . In larger panels, sparse mobility and strong sorting create the same structure at scale, with cheap diagonal blocks but cross-factor blocks that determine how difficult the fixed-effects problem is.

Flipping the sign of the worker-firm cross-tabulation turns the worker-firm sub-block of G into a graph Laplacian,

$$L_{WF} = \begin{pmatrix} 2 & 0 & 0 & -1 & -1 \\ 0 & 2 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & -2 \\ -1 & -2 & 0 & 3 & 0 \\ -1 & 0 & -2 & 0 & 3 \end{pmatrix}, \quad (15)$$

L_{WF} satisfies the defining Laplacian properties: it is symmetric (inherited from G); the diagonal entry $G_{WW}[i, i]$ counts worker i 's observations, which equals $\sum_f C_{WF}[i, f]$, the weighted degree of i in the bipartite worker-firm graph (and analogously for firms); and the sign flip makes the off-diagonal entry between worker i and firm f equal to $-C_{WF}[i, f]$, the negative edge weight. Row sums are zero, and

positive semidefiniteness follows by writing L_{WF} as a sum of rank-one edge contributions. The same construction works for any factor pair, and Section 7 exploits it.

Whether G admits a closed-form residualization is decided by the panel design. In a fully balanced panel, where every (worker, firm), (worker, year), and (firm, year) pair is observed exactly once (as in a hypothetical mandatory rotation program), all off-diagonal blocks of G are rank-one outer products of ones vectors. The three projections then commute, and the FWL residual collapses to a closed-form generalization of the textbook double-demeaning formula (Baltagi 2021). Real worker-firm panels never look like this: C_{WF} is sparse and irregular, the projections do not commute, and no one-pass residual exists. The coupled system Equation 7 must therefore be solved iteratively, which is what we turn to next.

5. Alternating Projections and Graph Connectivity

The workhorse algorithm for multi-way fixed effects is the Method of Alternating Projections (MAP), also known as iterative demeaning or the “zig-zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013). Many packages use MAP or variants of it, often with accelerations (Berge 2018; Correia 2017); the standalone appendix describes the `fixest` acceleration strategy. While sections 3 and 4 gave a picture and an algebra for the fixed-effect graph, this section introduces the method of alternating projections and asks how the geometry of the fixed effects graph governs MAP’s convergence rate.

In a nutshell, MAP solves the FWL residualization problem by updating one fixed-effect dimension at a time. In the worker-firm-year model, worker means are removed from the current residual, then firm means are removed from the updated residual, then year means are removed, and we iterate until convergence.

Writing the FWL normal equations Equation 7 in block form, with $D = [D_W \ D_F \ D_Y]$, gives

$$\begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix} \begin{pmatrix} \alpha_W \\ \alpha_F \\ \alpha_Y \end{pmatrix} = \begin{pmatrix} D_{W'} W \mu \\ D_{F'} W \mu \\ D_{Y'} W \mu \end{pmatrix}. \quad (16)$$

Equivalently,

$$G_{WW}\alpha_W + C_{WF}\alpha_F + C_{WY}\alpha_Y = D_{W'} W \mu, \quad (17)$$

$$C_{(WF)'}\alpha_W + G_{FF}\alpha_F + C_{FY}\alpha_Y = D_{F'} W \mu, \quad (18)$$

$$C_{(WY)'}\alpha_W + C_{(FY)'}\alpha_F + G_{YY}\alpha_Y = D_{Y'} W \mu. \quad (19)$$

If the firm effects α_F and year effects α_Y were known, the first equation would give the worker effects α_W from

$$G_{WW}\alpha_W = D_{W'} W (\mu - D_F \alpha_F - D_Y \alpha_Y). \quad (20)$$

If the worker effects α_W and year effects α_Y were known, the second equation would give the firm effects α_F from

$$G_{FF}\alpha_F = D_{F'} W (\mu - D_W \alpha_W - D_Y \alpha_Y). \quad (21)$$

The year equation is analogous.

The core insight behind MAP is that G_{WW} , G_{FF} , and G_{YY} are diagonal matrices of weighted group counts. Therefore each block solve, conditional on the other factors, is cheap: it is just a group-mean calculation. We do not need a general matrix inverse for these blocks. If C is diagonal with nonzero entries c_j , then solving $Cz = b$ is simple elementwise division

$$z_j = \frac{b_j}{c_j}. \quad (22)$$

Applied to G_{WW} , this divides each worker-level weighted residual sum by the worker's weighted observation count; the firm and year updates are analogous.

MAP uses this fact iteratively, solving the worker block using the current firm and year effects, updating the residual, and then solving the firm block and year block in turn. One sweep applies this logic factor by factor. Equivalently, each update subtracts the weighted group mean of the current partial residual for the active fixed-effect dimension.

This gives MAP a standard interpretation in numerical linear algebra: it is block Gauss-Seidel on the fixed-effect normal equations.

The cross-tabulation blocks C_{WF} , C_{WY} , and C_{FY} enter this algorithm only indirectly. For example, the worker update is computed from the partial residual $\mu - D_F\alpha_F - D_Y\alpha_Y$, while the firm update is computed from $\mu - D_W\alpha_W - D_Y\alpha_Y$. Thus worker-firm, worker-year, and firm-year links are not solved as coupled subproblems; their effect is carried through the residual that one block update passes to the next.

This is effective when the graph is well connected. In a high-mobility worker-firm panel, many workers move across firms, so worker and firm effects are compared through many overlapping employment histories. A high wage at one firm can be related to wages of the same workers at other firms, and a worker update quickly changes the information seen by the next firm update, and vice versa.

When mobility is sparse, sorting is strong, or one factor is nearly nested in another, the same issue becomes both an identification problem and a numerical problem. If a worker is observed only at one firm, a high wage is hard to attribute: it may reflect a high worker effect, a high firm effect, or both. Movers provide the links that separate these explanations. When those links are few or concentrated in narrow parts of the graph, the cross-factor information that distinguishes worker effects from firm effects is weak. MAP can still solve the linear system, but it accesses this information only through repeated residual updates. It can therefore require many fast demeaning steps before the worker and firm effects are separated.

This intuition has a precise quantitative counterpart. We summarize MAP-hardness for a factor pair (q, r) with a single number, the spectral gap $1 - \rho_{qr}$. Here $\rho_{qr} = \cos^2(\theta_F)$ is the squared cosine of the Friedrichs angle between the q - and r -subspaces, computed from the second singular value of the normalized cross-tabulation $H_{qr} = N_q^{-\frac{1}{2}}C_{qr}N_r^{-\frac{1}{2}}$. A gap near zero means MAP contracts very slowly along the (q, r) pair (sparse mobility, near-nested structure); a gap near one means it contracts quickly. For worker-firm-year specifications the year pair is usually easy, and the hard geometry sits in the worker-firm pair. With three or more absorbed factors this is a pairwise diagnostic on the full cyclic MAP rate, not a bound; the standalone appendix gives the precise statement.

6. Graph Preconditioning for Fixed Effects

Section 5 attributed MAP’s slow convergence to thin connections in the fixed-effect graph. The same sparsity makes the Gramian G poorly conditioned, so any iterative method applied directly to G converges slowly. Preconditioning is the standard remedy for poorly conditioned iterative solves: rather than change the system $G\alpha = b$, one solves the equivalent system

$$M^{-1}G\alpha = M^{-1}b \quad (23)$$

with a matrix M chosen so that $M^{-1}G$ has better geometry than G . The fitted values are unchanged; only the operator the iteration sees is changed.

The ideal choice $M = G$ converges in a single step, but inverting the Gramian and computing G^{-1} is exactly the costly operation we are trying to avoid when applying the Frisch-Waugh-Lovell theorem. A useful preconditioner must therefore approximate G^{-1} closely enough to reduce the number of required iterations substantially, while remaining cheap enough to construct and apply that the gain is not lost to setup and per-iteration overhead.

In the fixed-effect setting at hand, both requirements point to the same design principle: build M^{-1} from simpler local subproblems that keep core geometric properties, but never the full system. In the simplest case, each subproblem involves only one fixed-effect dimension (workers, firms, or years). The corresponding block of G is diagonal, listing weighted observation counts per level, so applying M^{-1} reduces to elementwise division. This is the diagonal preconditioner used by `FixedEffectModels.jl` together with LSMR (Fong and Saunders 2011; `FixedEffectModels.jl` Developers 2026). A richer subproblem uses two factors jointly and incorporates the worker-firm, worker-year, and firm-year co-occurrence blocks C_{qr} that diagonal scaling discards; Section 7 instantiates this construction as the factor-pair Schwarz preconditioner.

6.1. Diagonal Preconditioning

Consider a labor market with a few very large firms and many small firms - Novo Nordisk in Danish register data, or Samsung in South Korean matched employer-employee panels. In the fixed-effect normal equations, a firm with tens of thousands of worker-year observations has a diagonal entry orders of magnitude larger than a five-person shop. A Krylov method that sees the unscaled system must make progress across directions measured on very different numerical scales. Scaling the columns of D by diagonal information from $D'WD$ makes these directions more comparable; this can therefore improve the convergence of a Krylov method like the LSMR, even though it does not use the worker-firm co-occurrence graph.

Diagonal scaling is therefore an important baseline, but it uses only count information from the diagonal of $D'WD$ and leaves the co-occurrence blocks out of the solver geometry. Including selected off-diagonal blocks C_{qr} without forming or solving the entire Gramian requires a preconditioner built from local factor-pair problems that remain much smaller than the global system while containing the co-occurrence structure between two fixed effects.

7. The Factor-Pair Schwarz Preconditioner

Diagonal preconditioning uses only the level counts in the diagonal blocks of the Gramian G . The preconditioner we propose here also incorporates information from selected off-diagonal blocks. These blocks record which levels of one factor co-occur with levels of another, and are therefore the algebraic representation of the fixed-effect graph. In the AKM case, C_{WF} records which workers connect which firms, so it contains the mobility structure that drives both identification and computational difficulty. MAP uses this information only indirectly, through repeated one-factor updates.

The worker-firm example panel from Section 4 illustrates the contrast. A worker-level MAP update inverts only the diagonal count matrix $N_W = \text{diag}(2, 2, 2)$. The associated inverse is cheap to compute, but the diagonal has no entries linking workers to firms. The worker-firm factor-pair block instead adds the firm counts and the cross-tabulation C_{WF} , so a local solve against this block uses the observed worker-firm edges directly.

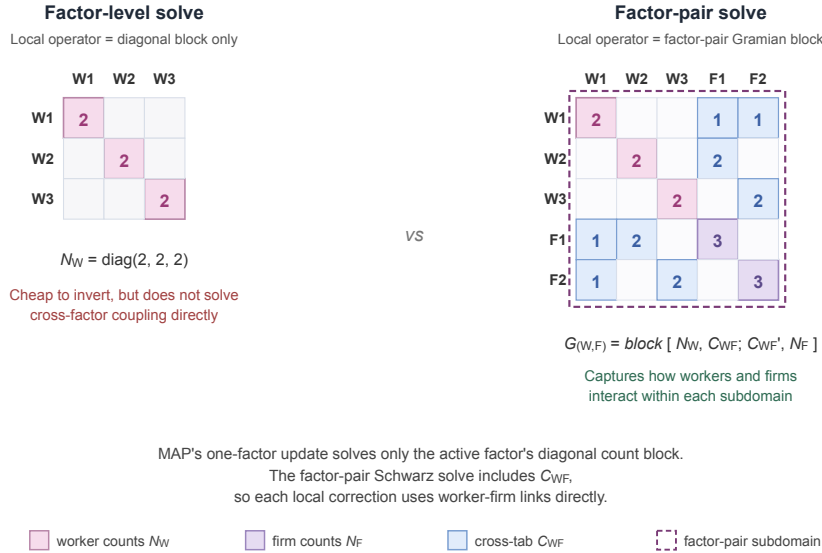


Figure 3: Local operator seen by a factor-level MAP update (left) versus the factor-pair Schwarz solve (right), shown on the toy worker-firm panel of Section 4. The factor-level block is the diagonal $N_W = \text{diag}(2, 2, 2)$. The factor-pair block adds the firm count block $N_F = \text{diag}(3, 3)$ and the cross-tabulation C_{WF} in its off-diagonal positions; the dashed outline marks the worker-firm subdomain that the local Schwarz solve operates on.

One could now try to invert these factor-pair blocks directly and use the exact local inverses inside the solver. That would use the right graph information, but it would also be too expensive for large pairs. The route we take is cheaper. After flipping the sign of one side, a factor-pair block becomes a graph Laplacian. This allows us to replace the exact local inverse by an approximate Laplacian solve, using the randomized approximate Cholesky ideas from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025).

The approximate local inverses are then collected in an additive Schwarz preconditioner. The Schwarz preconditioner is not the estimator itself; it is a reusable linear operator applied inside a Krylov solver.

This is why the approximation is not a change in the econometric target. CG still solves the original fixed-effect normal equations; the preconditioner only supplies better search directions. If a local inverse is imperfect, the Krylov iteration corrects the remaining error and stops only when the global residual reaches the requested tolerance. Once the preconditioner has been built, the same object can be reused when residualizing the outcome and each covariate.

Operationally, the construction proceeds in five steps. Form factor-pair problems such as worker-firm, worker-year, and firm-year. Split each pair graph into connected components. Sign-flip one side of each pair to obtain a local Laplacian. Approximate the local inverse rather than forming it exactly. Finally, combine the local approximations in an additive Schwarz preconditioner and apply it inside CG.

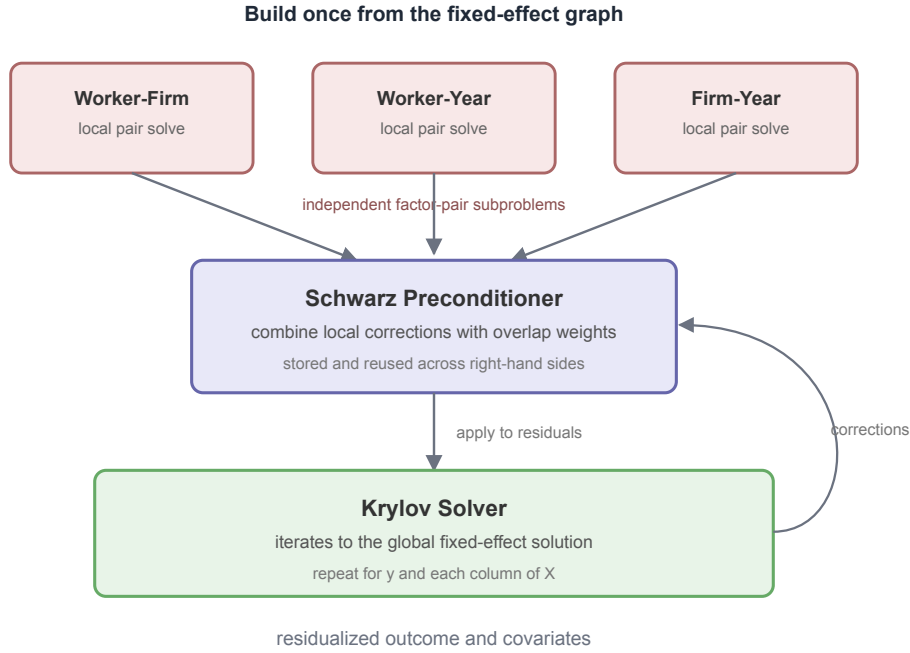


Figure 4: Macro strategy of the factor-pair preconditioner. Local pair solves are built first, combined into a reusable Schwarz preconditioner, and then applied repeatedly inside the Krylov solver for the outcome and covariates.

For notation, fix a pair of factors (q, r) . Let $N_q = G_{qq}$ and $N_r = G_{rr}$ be the diagonal weighted-count blocks for the two factors. These are the same kind of blocks as G_{WW} , G_{FF} , and G_{YY} in the AKM Gramian above; we write N to emphasize that they are count matrices. Let C_{qr} denote the cross-tabulation block. The local factor-pair block is

$$G_{qr} = \begin{pmatrix} N_q & C_{qr} \\ C_{(qr)'} & N_r \end{pmatrix}. \quad (24)$$

The preconditioner follows the additive Schwarz view of subspace correction methods (Xu 1992; Toselli and Widlund 2005). The default implementation enumerates all unordered factor pairs, builds C_{qr} for each pair, splits the induced bipartite graph into connected components, and creates one Schwarz subdomain per component. In a worker-firm-year AKM specification this gives worker-firm, worker-year, and firm-

year subdomains. The worker-firm pair is often the difficult one, but the construction includes all pairs by default.

For a subdomain s , let R_s restrict a global vector to the fixed-effect levels in that subdomain, and let A_s be the corresponding local factor-pair operator. If a fixed-effect level j appears in c_j local subdomains, assign weight $\omega_j = \frac{1}{\sqrt{c_j}}$ on both restriction and prolongation. With $\tilde{D}_s$ collecting these weights, the additive Schwarz preconditioner applied to a residual r is

$$M^{-1}r = \sum_{s=1}^{N_s} R_s' \tilde{D}_s A_s^+ \tilde{D}_s R_s r, \quad (25)$$

where the sum runs over the N_s Schwarz subdomains and A_s^+ denotes an approximate local solve on the normalized local subspace. Using the same weights on restriction and prolongation makes the additive preconditioner symmetric, so it can be paired with conjugate gradient (CG).

The Gramian G is symmetric positive semi-definite and singular. Within each connected component of the factor-interaction graph, a constant can be shifted between factors without changing $D\alpha$, leaving a one-dimensional null space per component. The right-hand side $D'W\mu$ lies in the column space of G , so the system is consistent. CG started from zero stays in that column space and converges to the minimum-norm solution, which is what FWL residualization requires.

Algorithm 1 summarizes the construction and one application of the preconditioner. The notation in the algorithm is as follows: R_s restricts a global coefficient-space vector to subdomain s ; A_s is the local factor-pair operator; A_s^+ is its approximate pseudoinverse on the normalized, zero-mean local subspace; and $\tilde{D}_s = \text{diag}(\omega_j)_{j \in s}$ contains the partition-of-unity weights, with $\omega_j = \frac{1}{\sqrt{c_j}}$ when level j appears in c_j subdomains.

Algorithm 1. Factor-Pair Schwarz Preconditioner

Inputs

- Observation-level factor codes for Q absorbed dimensions.
- Diagonal weights W and a local solver configuration.
- Krylov residual r in coefficient space.

Preconditioner setup

- Enumerate all unordered factor pairs (q, r) with $q < r$.
- For each pair, build weighted count blocks N_q, N_r and the weighted cross-tabulation C_{qr} .
- Split the induced bipartite graph into connected components and create one Schwarz subdomain s per component.
- If fixed-effect level j appears in c_j subdomains, store the partition weight $\omega_j = \frac{1}{\sqrt{c_j}}$.
- For each subdomain, sign-flip one side to obtain a local Laplacian, project the local right-hand side off the component constant, and build a zero-mean pseudoinverse solve.
- Use a Schur-complement local solver: small reduced systems are solved directly, while larger reduced SDD/Laplacian systems are solved with randomized approximate Cholesky.

Krylov application

- Initialize $z = 0$.
- For each subdomain s , form $h_s = \tilde{D}_s R_s r$.
- Compute the approximate local correction $u_s \approx A_s^+ h_s$ on the normalized subspace.
- Accumulate $z \leftarrow z + R_s' \tilde{D}_s u_s$.

- Return $z = M^{-1}r$.

8. Benchmarks

The preceding sections make a computational prediction: graph preconditioning should not beat MAP everywhere, but it should help precisely when the fixed-effect graph makes one-factor demeaning transmit information slowly. The benchmarks test this prediction in settings that move from controlled synthetic stress tests to standard public benchmark datasets.

The benchmarks address three practical questions. First, do the runtime gains appear in the regression interfaces that applied users actually call? The algorithmic difference is in the fixed-effect absorption step, but benchmarking only a demeaning kernel would omit software costs that matter in practice: parsing the model, constructing fixed-effect encodings, preparing right-hand sides, reusing solver state across variables, and returning the coefficient estimate. Second, is the memory overhead of storing factor-pair information small enough to keep the method usable on ordinary machines? Third, does the new solver return the same least-squares solution as MAP up to numerical tolerance, even though the implementations use different convergence checks and stopping thresholds?

For this reason, and to remain comparable with the public PyFixest benchmark suite, the runtime benchmarks use the package-level regression APIs. Each runtime includes model setup, construction of the fixed-effect representation, residualization of the outcome and covariates, and estimation of the coefficient of interest. The results should be read as software-level timings for the same econometric problem, not as isolated timings of the demeaning routine alone.

To connect timings back to the graph story, the runtime tables also report a compact hardness heuristic. For the relevant factor pair, we compute $\rho_{qr} = \sigma_2(H_{qr})^2$ after removing the component-wise constant singular direction, and report the gap $1 - \rho_{qr}$ together with the observation share of the component attaining the reported value. Smaller gaps mean harder two-factor MAP geometry; a large component share means that the hard geometry is not confined to a negligible part of the sample. With three or more fixed effects, these are pairwise diagnostics, not full convergence bounds.

Our benchmarks are organized in three steps. First, we use controlled AKM synthetic datasets that mimic worker-firm mobility and sorting patterns to isolate the mechanism behind the theory: MAP slows when connectivity weakens, while a factor-pair preconditioned Krylov method remains stable. Second, we move to standard synthetic benchmarks: the simple/difficult DGPs from the fixest benchmark suite and the synthetic benchmark datasets currently collected by Sergio Correia. Third, we use real datasets from the same Correia collection, because synthetic DGPs can miss the irregularities of real networks.

We compare the new preconditioned implementation to three existing solver strategies: a vanilla MAP implementation, a mature accelerated MAP implementation, and a diagonally-preconditioned Krylov implementation. The benchmarked backends are:

Backend	Package	Algorithm
rust-map	PyFixest	Vanilla Rust MAP backend without acceleration.

Backend	Package	Algorithm
fixest	R fixest	Sophisticated MAP implementation with Irons-Tuck acceleration, coefficient-space routines, and additional heuristics (Bergé et al. 2026); see the standalone appendix.
FEM.jl	FixedEffectModels.jl	LSMR, a Krylov method with diagonal preconditioning (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026).
within	PyFixest	Conjugate-gradient backend with factor-pair Schwarz preconditioning.

All reported CPU numbers are medians across three benchmark iterations run on an Apple M4 Mac mini with 10 CPU cores and 16 GB memory under macOS 15.3.1.

We do not include `reghdfe` directly in the benchmark tables because Stata is not open source and we do not have a licensed installation. `reghdfe` is a mature accelerated-MAP implementation with a conjugate-gradient option, singleton-observation pruning, and accelerated demeaning (Correia 2014; 2017); algorithmically it sits in the same family as `fixest`’s accelerated MAP.

8.1. Runtime Benchmarks

8.1.1. Controlled Synthetic Benchmarks: AKM Mobility and Sorting

We begin with controlled experiments on synthetic datasets that mimic key AKM properties. Each experiment varies one graph feature while holding the rest of the data-generating process fixed, so changes in runtime can be attributed to graph geometry.

The first experiment decreases worker mobility. Ex ante, the theory predicts that MAP should become slower as mobility falls, because one-factor demeaning has fewer worker moves through which to transmit information across firms. The preconditioned Krylov algorithm should therefore become more competitive in the low-mobility designs, where its factor-pair preconditioner can use the worker-firm graph directly.

Mobility benchmark ($n = 1\text{M}$).

Scenario	Gap (share)	rust-map	fixest	FEM.jl	within
akm_mobility_1	1.38×10^{-1} (1.00)	0.29s	0.16s	0.28s	0.51s
akm_mobility_2	3.41×10^{-2} (1.00)	1.10s	0.36s	0.58s	0.38s
akm_mobility_3	5.18×10^{-3} (0.61)	12.48s	1.31s	1.80s	0.33s
akm_mobility_4	3.04×10^{-4} (0.27)	51.95s	3.42s	2.78s	0.34s
akm_mobility_5	1.65×10^{-3} (0.53)	63.23s	4.17s	3.34s	0.35s
akm_mobility_6	7.57×10^{-4} (0.37)	–	5.27s	3.86s	0.35s

Note: AKM-style panel with 1M observations, one covariate, and worker, firm, and year fixed effects. Moving down the table lowers worker mobility in a 10-period panel, thinning the worker-firm graph. Lower mobility makes the worker-firm pair harder for MAP and relatively more favorable to the preconditioned method. Gap is $1 - \rho_{WF}$, with $\rho_{WF} = \cos^2(\theta_F)$ for the worker-firm pair; parentheses give the observation share of the component attaining the gap. Hardness diagnostics are computed on representative 100K draws with the same DGP knobs.

The mobility benchmark follows the predicted pattern. When mobility is high, preconditioning is not especially useful: `fixest` is fastest in `akm_mobility_1`, and `within` pays setup costs that are not yet offset by faster convergence. As mobility falls, the worker-firm gap shrinks from 1.38×10^{-1} to below 10^{-3} , and MAP runtimes increase sharply. `within` stays between 0.33s and 0.51s. In the lowest-mobility designs,

the preconditioned method is the fastest backend because it uses the worker-firm graph directly rather than transmitting information through many one-factor sweeps.

The second experiment increases sorting of workers to firms. Stronger sorting makes the worker-firm graph closer to block diagonal: movers connect firms mostly within similar groups rather than across distant parts of the graph. The same theory predicts that MAP should again lose ground as sorting rises, while the preconditioned method should be less sensitive because its local factor-pair solves capture this cross-factor structure.

Sorting benchmark ($n = 1\text{M}$).

Scenario	Gap (share)	rust-map	fixest	FEM.jl	within
akm_sorting_1	7.62×10^{-3} (0.83)	6.79s	0.81s	1.43s	0.32s
akm_sorting_2	2.41×10^{-3} (0.62)	9.73s	1.12s	1.68s	0.34s
akm_sorting_3	1.50×10^{-4} (0.59)	10.31s	1.24s	1.67s	0.33s
akm_sorting_4	2.14×10^{-5} (0.51)	14.94s	1.58s	1.87s	0.33s
akm_sorting_5	3.97×10^{-5} (0.53)	25.77s	1.80s	1.94s	0.34s

Note: AKM-style panel with 1M observations, one covariate, and worker, firm, and year fixed effects. Moving down the table increases sorting among movers, making the worker-firm graph closer to block diagonal. Stronger sorting weakens cross-block information flow and raises the value of factor-pair preconditioning. Gap is $1 - \rho_{WF}$ for the worker-firm pair; parentheses give the observation share of the component attaining the gap. Hardness diagnostics are computed on representative 100K draws with the same DGP knobs.

The sorting benchmark shows the same mechanism from a different direction. As movers become more sorted across firms, the graph approaches separated blocks, the worker-firm gap falls by two orders of magnitude, and MAP-based runtimes rise. within remains essentially flat, between 0.32s and 0.34s, because the factor-pair preconditioner already encodes the worker-firm coupling that sorting makes hard for one-factor residual updates to discover.

8.1.2. Standard Synthetic Benchmarks: fixest DGPs + Correia Synthetic

The AKM benchmarks above are designed to isolate a mechanism. We next turn to synthetic benchmark datasets that have become useful reference cases in fixed-effect software: they are public, easy to reproduce, and already used to compare implementations. Results on these datasets are therefore informative even when they are not tailored to the particular graph features emphasized by this paper.

The first family is the simple-versus-difficult DGP benchmark from `fixest` (Bergé et al. 2026), also used in the PyFixest benchmark suite (PyFixest Developers 2026). Both designs use 10M observations, one covariate, and three fixed effects (worker, firm, year). The simple design has dense random mobility; the difficult design has sparse, nearly nested worker-firm structure. We would therefore expect the simple design to be simple for MAP, because one-factor demeaning can move information quickly through a well-connected graph. Conversely, the difficult design should be difficult for MAP because worker and firm effects are close to collinear. Graph preconditioning should look good on the difficult design, but may not amortize its setup cost on the simple design.

Simple vs. difficult design (10M observations, 3 FE).

Design	Gap (share)	rust-map	fixest	FEM.jl	within	torch-cuda
simple (dense graph)	8.57×10^{-1} (1.00)	2.01s	0.93s	2.27s	10.53s	4.73s
difficult (sparse graph)	1.67×10^{-5} (1.00)	382.9s	32.7s	28.7s	3.25s	8.73s

Note: Medians over three full regression calls. Both designs use 10M observations, one covariate, and three fixed effects. Gap is $1 - \rho_{WF}$ for the worker-firm pair, reported from the generated 1M version of the same DGP family; the runtime rows use the same simple/difficult graph construction at 10M observations. `torch-cuda` is the PyFixest GPU LSMR backend with diagonal preconditioning (the same algorithm as `FixedEffectModels.jl`) run on an NVIDIA CUDA device; the local benchmark machine does not have a CUDA GPU. Standalone within demeaning API on these designs splits one-shot runtime into reusable solver construction and batch solve: simple design 5.76s + 1.51s (about 75% setup); difficult design 0.40s + 1.00s (about 29% setup). PyFixest regression overhead is added on top.

The fixest DGPs show the same tradeoff in its cleanest form. On the simple design, where the graph is dense and the worker-firm gap is large, `fixest` with Irons-Tuck acceleration is fastest at 0.93s and `within` is slowest at 10.53s; the preconditioner setup cost is not repaid when one-factor demeaning already converges quickly. As the caption notes, about three quarters of `within`'s one-shot demeaning time on the simple design is reusable solver setup, not the batch solve itself.

On the difficult design, the ranking reverses. MAP convergence degrades sharply: `rust-map` without acceleration does not finish in under six minutes, and `fixest` takes 32.7s, while `within` finishes in 3.25s because its factor-pair preconditioner captures sparse worker-firm coupling directly. The diagnostic gap is almost zero, so the table is measuring the regime in which MAP should need many sweeps to separate worker and firm effects. The setup share of the standalone demeaning time also falls to about 29%. The difficult design is exactly where pair information helps; the simple design is easy enough that building the graph preconditioner is a cost rather than a shortcut.

The GPU backend (`torch-cuda`) runs the same diagonally preconditioned LSMR algorithm as `FixedEffectModels.jl`, but on an NVIDIA CUDA device. On the simple design it is not competitive with CPU `fixest` (4.73s vs. 0.93s): host-to-device transfer and kernel launch overheads outweigh the gain from parallelizing already cheap iterations. On the difficult design, GPU parallelism brings runtime down to 8.73s - roughly three times faster than CPU `FEM.jl` - but the iteration count is still governed by the diagonal preconditioner's quality, which on this graph is limited. `within` at 3.25s remains faster on CPU by using a factor-pair preconditioner that captures the cross-factor coupling the diagonal cannot. Hardware acceleration and stronger preconditioning attack different bottlenecks; on poorly conditioned designs the preconditioner is the larger lever.

The second family of benchmarks consists of synthetic datasets from the Correia HDFE benchmark collection. These datasets span a wider set of graph shapes: complete bipartite matching, uniform random matching with varying connectivity, assortative matching, and a small path-like design. They provide an independent public reference set for comparing the same software backends outside the AKM generator.

Correia synthetic benchmarks.

Dataset	Gap (share)	<code>rust-map</code>	<code>fixest</code>	<code>FEM.jl</code>	<code>within</code>
synthetic-complete	1.000 (1.00)	0.074s	0.062s	0.044s	0.114s
synthetic-uniform-easy	0.651 (1.00)	0.122s	0.079s	0.083s	0.117s
synthetic-uniform-hard	0.184 (1.00)	0.367s	0.251s	0.353s	0.957s
synthetic-uniform-harder	0.0249 (1.00)	0.994s	0.952s	0.575s	0.514s
synthetic-assortative	0.00133 (0.70)	26.22s	3.02s	2.99s	1.46s

Note: Medians over three runs from the `singleton_drop = true` rows of `pyfixest/benchmarks/results/correia-benchmarks.csv`. Gap is $1 - \rho$ for the `id1-id2` pair after the same singleton pruning; parentheses give the observation share of the component attaining

the gap. Smaller gaps indicate slower two-way MAP geometry. The synthetic-zigzag dataset is omitted: every backend terminates with a numerical error rather than a converged solution under default settings, so it is a stress case rather than a runtime comparison.

These results are less clean than the AKM experiments, which is useful in itself. Several datasets are small enough, or well connected enough, that setup cost matters as much as conditioning; on the complete and easier uniform designs, the gap is large and low-overhead methods do well. The harder rows look different. By synthetic-uniform-harder, the gap has fallen to 2.49×10^{-2} and within is already the fastest backend. On the assortative benchmark, the gap is roughly 1.33×10^{-3} and the preconditioned method has its clearest advantage: sorting creates cross-factor structure that one-factor updates handle slowly.

8.1.3. Standard Real-Data Benchmarks: Correia Collection

We then move to real benchmark networks from the Correia collection. These are not only larger or noisier; they contain empirical graph irregularities that synthetic generators only approximate: degree heterogeneity, sparse bridges, component structure, and identifier interactions beyond the canonical two-way pair.

Correia real-data benchmarks.

Dataset	Gap (share)	rust-map	fixest	FEM.jl	within
credit	0.402 (1.00)	0.177s	0.134s	0.158s	0.139s
soccer	0.889 (1.00)	0.018s	0.014s	0.016s	0.031s
enron	0.00704 (0.98)	4.37s	0.749s	0.689s	0.537s
github	0.000630 (0.32)	86.44s	4.50s	4.60s	0.380s
patents	0.000518 (0.95)	19.66s	1.29s	1.43s	0.826s
workers	0.000274 (0.63)	128.34s	5.55s	4.94s	0.491s
schools	0.00221 (1.00)	11.04s	1.10s	1.28s	0.224s
directors	0.000512 (0.30)	4.26s	0.216s	1.21s	0.300s

Note: Medians over three runs from the `singleton_drop = true` rows of `pyfixest/benchmarks/results/correia-benchmarks.csv`. Gap is $1 - \rho$ for the `id1-id2` pair after the same singleton pruning; parentheses give the observation share of the component attaining the gap. A tiny gap with a limited component share, as in `directors`, signals a hard subcomponent but not necessarily whole-sample MAP difficulty.

The real datasets make the same lesson less stylized. Accelerated MAP is hard to beat on small or compact graphs such as `credit` and `soccer`, where the gaps are large. The `directors` row shows why the component share is useful: the worst component is hard, but it contains about thirty percent of observations, so the whole problem is not as costly for MAP as the gap alone would suggest. On larger networks with hard components covering a substantial part of the sample, especially `enron`, `github`, `patents`, `workers`, and `schools`, the factor-pair preconditioner is fastest by a wide margin. Synthetic designs are useful for isolating why this happens; real networks show that the hard graph features also appear in ordinary benchmark data.

8.2. Memory Use

One main advantage of MAP is its small memory footprint: each iteration only needs the current residuals and fixed-effect level summaries. A graph preconditioner necessarily stores additional factor-pair information, so the main memory question is narrow: how much extra memory does `within` require relative to the Rust MAP backend?

We measure peak resident set size (peak RSS), the largest amount of physical memory used by the process during the run, on the simple and difficult fixed-effect DGPs. These are representative probes rather than special memory cases: the dominant storage terms are the data matrix, fixed-effect encodings, and reusable factor-pair objects, so results should mostly generalize across datasets with similar dimensions. We do not use this section to compare against `fixest` or `FixedEffectModels.jl`: cross-language peak RSS also reflects R and Julia runtime overhead, data-loading choices, garbage collection, and package internals. The diagnostic here is the incremental memory cost of replacing Rust MAP by the preconditioned Rust backend inside the same Python package. Since the surrounding regression code is shared, this comparison isolates the difference in demeaning strategies. Both backends are run in isolated processes and report peak RSS via `ru_maxrss`.

Memory footprint (3 FE, $k = 10$).

Design	Gap (share)	rust-map	within
<i>100K observations</i>			
simple (dense graph)	0.857 (1.00)	428 MB	487 MB
difficult (sparse graph)	0.00130 (1.00)	432 MB	479 MB
<i>1M observations</i>			
simple (dense graph)	0.857 (1.00)	1,188 MB	1,703 MB
difficult (sparse graph)	1.67×10^{-5} (1.00)	1,263 MB	1,398 MB

Note: Peak RSS means peak resident set size, measured from isolated Python processes. 10 covariates, three fixed effects. These two DGPs are representative probes; we do not claim memory behavior is design-specific. Gap is $1 - \rho_{WF}$ for the worker-firm pair in the corresponding generated design.

At 100K observations, the preconditioner adds roughly 50 MB in both an easy and a hard graph. At 1M, the overhead is larger in absolute terms (135–515 MB) but remains modest relative to the data footprint of a panel with 10 covariates. This is the expected tradeoff: `within` uses more memory than MAP, but the additional storage for factor-pair co-occurrences, partition weights, and local approximate Cholesky factors is still lightweight relative to the full regression problem.

8.3. Numerical Equivalence

When introducing a new fixed-effect solver, the burden is on the implementation to show that it matches existing solutions. Exact agreement should not be taken for granted: MAP, CG, and LSMR-style routines use different convergence checks, residual norms, stopping thresholds, and iteration caps. PyFixest therefore tests its solvers internally and checks package output against external implementations such as R `fixest` and Julia `FixedEffectModels.jl`. The tables below add targeted diagnostics for the PyFixest Rust backends used in this paper and for coefficient agreement with external software.

We use the 100K-observation simple and difficult designs as diagnostic cases. The simple design checks the easy regime where both methods should agree almost exactly. The difficult design checks the regime where conditioning is poor and small differences in stopping rules are most likely to be amplified. The graph diagnostic separates these cases sharply: the worker-firm gap is about 0.857 in the simple design and about 0.00130 in the difficult design. The table below collects slope coefficients and cross-software differences for all four backends. Additional within-PyFixest residual and fixed-effect diagnostics, a tolerance-scaling experiment, and the exact convergence rules used by each backend are reported in the

standalone appendix. The table should therefore be read as a default-settings agreement check, not as a common-tolerance accuracy contest.

Coefficient agreement (100K observations, 3 FE, $k = 10$).

Design	Backend	$\hat{\beta}_1$	Avg diff	Max diff
simple	rust-map	0.99914206	–	–
	within	0.99914206	1.5×10^{-14}	3.5×10^{-14}
	fixest	0.99909173	2.2×10^{-5}	5.0×10^{-5}
	FEM.jl	0.99914206	2.1×10^{-12}	5.3×10^{-12}
difficult	rust-map	1.00086122	–	–
	within	1.00086098	1.4×10^{-7}	3.4×10^{-7}
	fixest	1.00081490	2.0×10^{-5}	5.0×10^{-5}
	FEM.jl	1.00086098	1.4×10^{-7}	3.4×10^{-7}

Note: $\hat{\beta}_1$ is the slope coefficient on `x1`. Differences are absolute slope-coefficient deviations from `rust-map`, averaged (Avg) and maximized (Max) across all 10 covariates. `within` is the PyFixest preconditioned Rust backend. `fixest` and `FixedEffectModels.jl` are called through their own regression APIs. The simple and difficult designs have worker-firm gaps 0.857 and 0.00130, respectively, so the table compares numerical agreement in both an easy and a hard graph regime. The largest cross-backend gap is at the fifth decimal, which is below the precision at which applied coefficients are typically reported.

The `within` and `FEM.jl` rows are almost identical to `rust-map` at the reported defaults. R `fixest` is also close, but not at machine precision: the largest coefficient gap is about 5×10^{-5} . This is unsurprising because the packages do not share a stopping rule. `fixest` stops on fixed-point coefficient changes, `FixedEffects.jl` uses LSMR backward-error tests, and `within` uses a relative residual in the fixed-effect normal equations. The standalone appendix records these rules explicitly and shows that the within-PyFixest gap closes as the tolerance tightens.

8.4. Poisson / PPML Benchmark

Benchmarks to be added.

9. Software Availability

The solver studied in this paper is available as open-source software through the `within` project (within Developers 2026a). The computational core is written in Rust and exposed through Rust, Python, and R interfaces; the language bindings call the same underlying solver. This matters for reproducibility and adoption: the method can be used directly from Rust, called from Python workflows, or used from R without reimplementing the algorithm.

Interface	Package	Registry	Install
Rust	<code>within</code>	crates.io (within Developers 2026b)	<code>cargo add within</code>
Python	<code>within-py</code>	PyPI (within Developers 2026c)	<code>pip install within-py</code>
R	<code>withinr</code>	py-econometrics R-universe (within Developers 2026d)	<code>install.packages("withinr", repos = ...)</code>

The Rust crate exposes the lower-level solver and configuration types. The Python and R packages expose solver-level `solve` and `solve_batch` style APIs for residualizing one or several right-hand sides. The Python package imports as `within` and is also integrated in PyFixest (PyFixest Developers 2026). The

high-level regression interfaces are still evolving, so we show the stable residualization API. A minimal call looks like

```
from within import CG, Solver, solve_batch
```

```
config = CG(tol=1e-6, maxiter=10_000)
```

```
solver = Solver(categories, config)
```

```
result = solver.solve_batch(rhs) # reuse setup across right-hand sides
```

```
# or one-shot:
```

```
result = solve_batch(categories, rhs, config)
```

where `categories` is a Fortran-ordered uint32 array of fixed-effect codes and `rhs` is a Fortran-ordered float64 matrix whose columns are the right-hand sides to residualize.

The R package can be installed from R-universe with

```
install.packages(  
  "withinr",  
  repos = c("https://py-econometrics.r-universe.dev",  
            "https://cloud.r-project.org")  
)
```

TODO: add a minimal `withinr` usage snippet once the R-side `solve` / `solve_batch` call signature is confirmed against the current `withinr` documentation.

In the benchmarks above, `within` uses its default local solver: Schur-complement reduction, dense solves for reduced systems with at most 24 levels, and randomized approximate Cholesky with `ApproxCholConfig(seed=0, split=1)` for larger reduced systems.

10. Conclusion

High-dimensional fixed effects are often presented as an econometric device for absorbing many categorical controls while estimating a lower-dimensional coefficient of interest. Computationally, the same specification defines a graph, and the connectivity of that graph affects how quickly information moves between factors during residualization.

MAP remains the right default when the graph is dense and well connected. Its iterations are cheap, mature implementations add strong accelerations, and a graph preconditioner may spend more time building structure than it saves. This is exactly what happens in the simple ‘fixest’ design and in small, compact Correia benchmarks.

The preconditioned solver is attractive when the hard part of the design is a sparse cross-factor pair. In AKM applications this is usually the worker-firm pair: low mobility, strong sorting, thin bridges between firm groups, or near nesting all make one-factor demeaning pass information slowly across the graph. Similar patterns arise in doctor-patient, student-teacher, exporter-importer, and product-market designs whenever the identifying moves are sparse or concentrated within clusters.

The practical diagnostic is therefore graph connectivity, not sample size alone. If MAP converges in a few sweeps, there is little to improve. If the fixed-effect pair has many small components, weak bridges,

mostly within-cluster movers, or a nearly nested structure, a factor-pair preconditioner gives the Krylov iteration information that MAP only sees after many residual updates.

Concretely, before running we recommend computing (a) the share of workers observed at more than one firm, (b) the size distribution of factor-pair connected components, and optionally (c) the second singular value of the normalized worker-firm overlap H_{WF} . A low mover share, many small components, or $\cos(\theta_F)$ close to one are all signals that MAP will be slow and the preconditioner will pay for itself.

The econometric target is unchanged; the gain comes from choosing a numerical path that better matches the geometry of the fixed effects.

Appendix: Local Factor-Pair Solver Details

This appendix gives the local-solver machinery summarised in Section 7. The local factor-pair solve works because each pair block has hidden Laplacian structure. For a pair (q, r) ,

$$G_{qr} = \begin{pmatrix} N_q & C \\ C' & N_r \end{pmatrix}. \quad (26)$$

After flipping the sign of one side, this becomes

$$L_{qr} = \begin{pmatrix} N_q & -C \\ -C' & N_r \end{pmatrix}. \quad (27)$$

L_{qr} is a valid weighted graph Laplacian: it is symmetric, its off-diagonal entries are non-positive, and its row sums are zero. The row-sum property holds because each observation contributes exactly one level on each side, so the diagonal count for level j of factor q equals the sum of its cross-tabulation row, $N_q[j, j] = \sum_k C_{qr}[j, k]$ (and symmetrically for r).

The sign flip can be written as a similarity transform. With $T = \begin{pmatrix} I & 0 \\ 0 & -I \end{pmatrix}$ one checks that $L_{qr} = TG_{qr}T$, and since $T^2 = I$, the system $G_{qr}z = h$ is equivalent to $L_{qr}u = Th$ with $z = Tu$. Because the Laplacian L_{qr} is singular on each connected component, the implementation uses the zero-mean component solution: the local right-hand side is projected off the constant direction, and the returned correction has mean zero on that component. The fitted correction is invariant to this normalization. The approximate Schur complement and approximate Cholesky steps, introduced next, can introduce small row-sum deficits that break the strict Laplacian property; when this happens the implementation applies *Gremban augmentation* (Gremban 1996), which adds a single grounded node connected to all others to absorb the deficit and restore a valid Laplacian on the augmented system.

The default local solver eliminates one side of the bipartite graph. If the eliminated side is q , the Schur complement is

$$S = N_r - C' N_q^{-1} C. \quad (28)$$

(The same Schur complement is obtained whether elimination is performed on G_{qr} or on the sign-flipped Laplacian L_{qr} , since $(-C')(-C) = C'C$.)

Exact elimination can create dense fill. In worker-firm language, eliminating a worker who visited d firms creates a clique among those firms with $\binom{d}{2}$ edges. For large factor pairs within therefore replaces this exact elimination with an *approximate* Schur step: each eliminated vertex's clique is replaced by a randomly sampled spanning tree on its neighbours, with $d - 1$ edges instead of $\binom{d}{2}$, and tree weights chosen so that the expected reduced Laplacian equals the exact Schur complement (an unbiased estimator) (Gao et al. 2025). The reduced system is then solved either by a dense factorization, when small, or by randomized approximate Cholesky on larger reduced SDD/Laplacian systems (Spielman and Teng 2014; Gao et al. 2025), which uses the same clique-tree sampling idea sequentially as it eliminates the remaining vertices. The local factors remain sparse and cheap to apply, while the outer Krylov iteration corrects the approximation error globally.

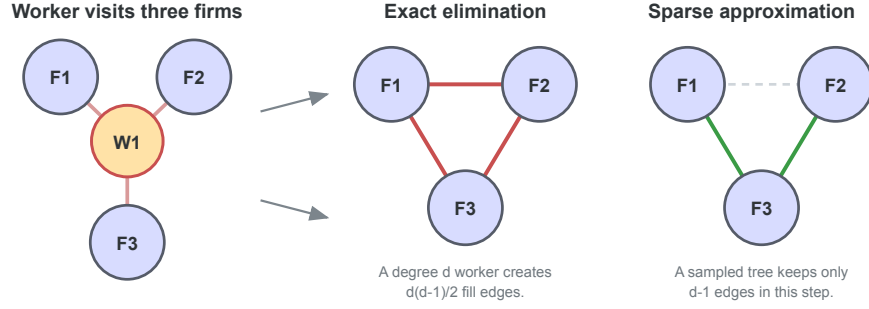


Figure 5: Approximate Schur step. Exact elimination of a vertex with d neighbours produces a clique with $\binom{d}{2}$ fill edges (left); the approximate variant replaces the clique with a randomly sampled spanning tree of $d - 1$ edges (right), with weights chosen so that the expected reduced Laplacian equals the exact Schur complement.

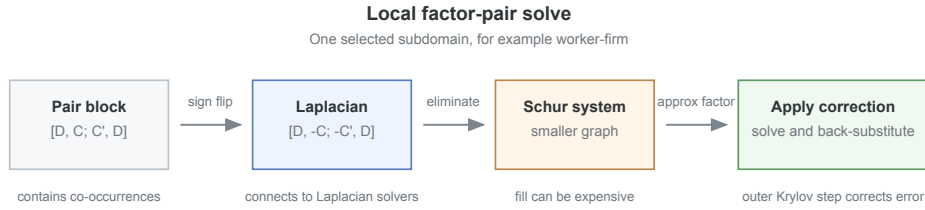


Figure 6: Local solve pipeline: sign flip, Schur reduction, approximate factorization, and back-substitution.

References

- Abowd, John M., Francis Kramarz, and David N. Margolis. 1999. “High Wage Workers and High Wage Firms.” *Econometrica* 67 (2): 251–333. <https://doi.org/10.1111/1468-0262.00020>.
- Baltagi, Badi H. 2021. *Econometric Analysis of Panel Data*. 6th ed. Springer.
- Berge, Laurent. 2018. *Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package Fenmlm*. Technical Report Nos. 2018–13.
- Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. “Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R.” <https://doi.org/10.48550/arXiv.2601.21749>.
- Card, David, Jörg Heining, and Patrick Kline. 2013. “Workplace Heterogeneity and the Rise of West German Wage Inequality.” *The Quarterly Journal of Economics* 128 (3): 967–1015. <https://doi.org/10.1093/qje/qjt006>.
- Chetty, Raj, John N. Friedman, and Jonah E. Rockoff. 2014. “Measuring the Impacts of Teachers II: Teacher Value-Added and Student Outcomes in Adulthood.” *American Economic Review* 104 (9): 2633–79. <https://doi.org/10.1257/aer.104.9.2633>.
- Correia, Sergio. 2014. “Reghdfe: Stata Module to Perform Linear or Instrumental-Variable Regression Absorbing Any Number of High-Dimensional Fixed Effects.” <https://github.com/sergiocorreia/reghdfe>.
- Correia, Sergio. 2017. *Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator*. Technical report. <https://hdl.handle.net/10.2139/ssrn.3129010>.

- FixedEffectModels.jl Developers. 2026. “Fixedeffectmodels.jl: Fast Estimation of Linear Models with High Dimensional Categorical Variables.”. <https://github.com/FixedEffects/FixedEffectModels.jl>.
- Fong, David Chin-Lung, and Michael Saunders. 2011. “LSMR: An Iterative Algorithm for Sparse Least-Squares Problems.” *SIAM Journal on Scientific Computing* 33 (5): 2950–71. <https://doi.org/10.1137/10079687X>.
- Frisch, Ragnar, and Frederick V. Waugh. 1933. “Partial Time Regressions as Compared with Individual Trends.” *Econometrica* 1 (4): 387–401. <https://doi.org/10.2307/1907330>.
- Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. “AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization.” *SIAM Journal on Scientific Computing*. <https://rasmuskyng.com/papers/GKS25.pdf>.
- Gaure, Simen. 2013. “OLS with Multiple High Dimensional Category Variables.” *Computational Statistics & Data Analysis* 66 : 8–18. <https://doi.org/10.1016/j.csda.2013.03.024>.
- Gremban, Keith D. 1996. “Combinatorial Preconditioners for Sparse, Symmetric, Diagonally Dominant Linear Systems.” Doctoral dissertation.
- Guimaraes, Paulo, and Pedro Portugal. 2010. “A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects.” *The Stata Journal* 10 (4): 628–49. <https://doi.org/10.1177/1536867X1101000406>.
- Head, Keith, and Thierry Mayer. 2014. “Gravity Equations: Workhorse, Toolkit, And Cookbook.” In *Handbook of International Economics*, edited by Gita Gopinath, Elhanan Helpman, and Kenneth Rogoff, vol. 4 of *Handbook of International Economics*. Elsevier. <https://doi.org/10.1016/B978-0-444-54314-1.00003-3>.
- Lovell, Michael C. 1963. “Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis.” *Journal of the American Statistical Association* 58 (304): 993–1010. <https://doi.org/10.1080/01621459.1963.10480682>.
- Molitor, David. 2018. “The Evolution of Physician Practice Styles: Evidence from Cardiologist Migration.” *American Economic Journal: Economic Policy* 10 (1): 326–56. <https://doi.org/10.1257/pol.20160319>.
- PyFixest Developers. 2026. “Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python.”. <https://github.com/py-econometrics/pyfixest>.
- Spielman, Daniel A., and Shang-Hua Teng. 2014. “Nearly Linear Time Algorithms for Preconditioning and Solving Symmetric, Diagonally Dominant Linear Systems.” *SIAM Journal on Matrix Analysis and Applications* 35 (3): 835–85. <https://doi.org/10.1137/090771430>.
- Toselli, Andrea, and Olof Widlund. 2005. *Domain Decomposition Methods: Algorithms and Theory*. Springer. <https://doi.org/10.1007/b137868>.
- within Developers. 2026b. “Within Rust Crate.”. <https://crates.io/crates/within>.
- within Developers. 2026c. “Within-Py: Python Bindings for Within.”. <https://pypi.org/project/within-py/>.
- within Developers. 2026a. “Within: High-Performance Fixed-Effects Solver for Econometric Panel Data.”. <https://github.com/py-econometrics/within>.
- within Developers. 2026d. “Withinr: R Bindings for Within.”. <https://py-econometrics.r-universe.dev/withinr>.

Xu, Jinchao. 1992. "Iterative Methods by Space Decomposition and Subspace Correction." *SIAM Review* 34 (4): 581–613. <https://doi.org/10.1137/1034116>.